

Dynamic Programming

Dr. Bugra Caskurlu¹

¹ School of Computing
New Uzbekistan University

October, 2025

What is Dynamic Programming?

A General Overview

- Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems.

What is Dynamic Programming?

A General Overview

- Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems.
- *Programming* in this context refers to a tabular method, not to writing computer code.

What is Dynamic Programming?

A General Overview

- Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems.
- *Programming* in this context refers to a tabular method, not to writing computer code.
- Divide-and-conquer algorithms partition the problem into *disjoint* subproblems, solve the subproblems recursively, and then combine their solutions to solve the original problem.

What is Dynamic Programming?

A General Overview

- Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems.
- *Programming* in this context refers to a tabular method, not to writing computer code.
- Divide-and-conquer algorithms partition the problem into *disjoint* subproblems, solve the subproblems recursively, and then combine their solutions to solve the original problem.
- In contrast, dynamic programming applies when the subproblems overlap. i.e., when the subproblems share subsubproblems.

What is Dynamic Programming?

A General Overview

- Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems.
- *Programming* in this context refers to a tabular method, not to writing computer code.
- Divide-and-conquer algorithms partition the problem into *disjoint* subproblems, solve the subproblems recursively, and then combine their solutions to solve the original problem.
- In contrast, dynamic programming applies when the subproblems overlap. i.e., when the subproblems share subsubproblems.
- In this context, a divide-and-conquer algorithm does more work than necessary, repeatedly solving the common subsubproblems.

What is Dynamic Programming?

A General Overview

- Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems.
- *Programming* in this context refers to a tabular method, not to writing computer code.
- Divide-and-conquer algorithms partition the problem into *disjoint* subproblems, solve the subproblems recursively, and then combine their solutions to solve the original problem.
- In contrast, dynamic programming applies when the subproblems overlap. i.e., when the subproblems share subsubproblems.
- In this context, a divide-and-conquer algorithm does more work than necessary, repeatedly solving the common subsubproblems.
- A dynamic-programming algorithm solves each subsubproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time it solves each subsubproblem.

What is Dynamic Programming?

A General Overview

- Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems.
- *Programming* in this context refers to a tabular method, not to writing computer code.
- Divide-and-conquer algorithms partition the problem into *disjoint* subproblems, solve the subproblems recursively, and then combine their solutions to solve the original problem.
- In contrast, dynamic programming applies when the subproblems overlap. i.e., when the subproblems share subsubproblems.
- In this context, a divide-and-conquer algorithm does more work than necessary, repeatedly solving the common subsubproblems.
- A dynamic-programming algorithm solves each subsubproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time it solves each subsubproblem.
- Dynamic programming applies to **optimization problems**. In such problems, each solution has a value, and you want to find a solution with the optimal (minimum or maximum) value.

What is Dynamic Programming?

A General Overview

- Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems.
- *Programming* in this context refers to a tabular method, not to writing computer code.
- Divide-and-conquer algorithms partition the problem into *disjoint* subproblems, solve the subproblems recursively, and then combine their solutions to solve the original problem.
- In contrast, dynamic programming applies when the subproblems overlap. i.e., when the subproblems share subsubproblems.
- In this context, a divide-and-conquer algorithm does more work than necessary, repeatedly solving the common subsubproblems.
- A dynamic-programming algorithm solves each subsubproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time it solves each subsubproblem.
- Dynamic programming applies to **optimization problems**. In such problems, each solution has a value, and you want to find a solution with the optimal (minimum or maximum) value.
- We call such a solution *an* optimal solution to the problem, as opposed to *the* optimal solution, since there may be several solutions that achieve the optimal value.

The Approach Taken in Dynamic Programming

The Steps of a Typical Dynamic Program

To develop a dynamic-programming algorithm, we follow a sequence of four steps:

The Approach Taken in Dynamic Programming

The Steps of a Typical Dynamic Program

To develop a dynamic-programming algorithm, we follow a sequence of four steps:

- 1 Characterize the structure of an optimal solution.

The Approach Taken in Dynamic Programming

The Steps of a Typical Dynamic Program

To develop a dynamic-programming algorithm, we follow a sequence of four steps:

- 1 Characterize the structure of an optimal solution.
- 2 Recursively define the value of an optimal solution.

The Approach Taken in Dynamic Programming

The Steps of a Typical Dynamic Program

To develop a dynamic-programming algorithm, we follow a sequence of four steps:

- 1 Characterize the structure of an optimal solution.
- 2 Recursively define the value of an optimal solution.
- 3 Compute the value of an optimal solution, typically in a bottom-up fashion

The Approach Taken in Dynamic Programming

The Steps of a Typical Dynamic Program

To develop a dynamic-programming algorithm, we follow a sequence of four steps:

- 1 Characterize the structure of an optimal solution.
- 2 Recursively define the value of an optimal solution.
- 3 Compute the value of an optimal solution, typically in a bottom-up fashion
- 4 Construct an optimal solution from computed information.

The Approach Taken in Dynamic Programming

The Steps of a Typical Dynamic Program

To develop a dynamic-programming algorithm, we follow a sequence of four steps:

- ❶ Characterize the structure of an optimal solution.
 - ❷ Recursively define the value of an optimal solution.
 - ❸ Compute the value of an optimal solution, typically in a bottom-up fashion
 - ❹ Construct an optimal solution from computed information.
- The first 3 steps form the basis of a dynamic-programming solution to a problem. If you need only the value of an optimal solution, and not the solution itself, then you can omit step 4.

The Approach Taken in Dynamic Programming

The Steps of a Typical Dynamic Program

To develop a dynamic-programming algorithm, we follow a sequence of four steps:

- ❶ Characterize the structure of an optimal solution.
 - ❷ Recursively define the value of an optimal solution.
 - ❸ Compute the value of an optimal solution, typically in a bottom-up fashion
 - ❹ Construct an optimal solution from computed information.
- The first 3 steps form the basis of a dynamic-programming solution to a problem. If you need only the value of an optimal solution, and not the solution itself, then you can omit step 4.
 - When you need to perform step 4, it often pays to maintain additional information during step 3 so that you can easily construct an optimal solution.

Weighted Interval Scheduling Problem

Problem Definition

- We have n requests labeled $1, 2, \dots, n$, with each request i specifying a *start time* s_i and a *finish time* f_i . Each interval i also has a value, or weight v_i .

Weighted Interval Scheduling Problem

Problem Definition

- We have n requests labeled $1, 2, \dots, n$, with each request i specifying a *start time* s_i and a *finish time* f_i . Each interval i also has a value, or weight v_i .
- Two intervals are *compatible* if they do not overlap.

Weighted Interval Scheduling Problem

Problem Definition

- We have n requests labeled $1, 2, \dots, n$, with each request i specifying a *start time* s_i and a *finish time* f_i . Each interval i also has a value, or weight v_i .
- Two intervals are *compatible* if they do not overlap.
- The goal of is to select a subset $S \subseteq \{1, 2, \dots, n\}$ of mutually compatible intervals, so as to maximize the sum of the values of the selected intervals, $\sum_{i \in S} v_i$.

Weighted Interval Scheduling Problem

Problem Definition

- We have n requests labeled $1, 2, \dots, n$, with each request i specifying a *start time* s_i and a *finish time* f_i . Each interval i also has a value, or weight v_i .
- Two intervals are *compatible* if they do not overlap.
- The goal of is to select a subset $S \subseteq \{1, 2, \dots, n\}$ of mutually compatible intervals, so as to maximize the sum of the values of the selected intervals, $\sum_{i \in S} v_i$.

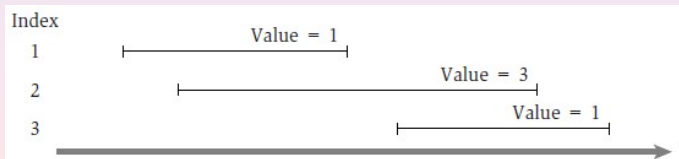


Figure: An Instance of Weighted Interval Scheduling Problem

Notation and Simplifying Assumptions

Introducing Notation and Terminology

- Suppose that the requests are sorted with respect to finish times: $f_1 \leq f_2 \leq \dots \leq f_n$.

Notation and Simplifying Assumptions

Introducing Notation and Terminology

- Suppose that the requests are sorted with respect to finish times: $f_1 \leq f_2 \leq \dots \leq f_n$.
- We'll say a request i comes before a request j if $i < j$. This will be the natural left-to-right order in which we'll consider intervals.

Notation and Simplifying Assumptions

Introducing Notation and Terminology

- Suppose that the requests are sorted with respect to finish times: $f_1 \leq f_2 \leq \dots \leq f_n$.
- We'll say a request i comes before a request j if $i < j$. This will be the natural left-to-right order in which we'll consider intervals.
- We define $p(j)$, for an interval j , to be the largest index $i < j$ such that intervals i and j are disjoint, i.e., i is the leftmost interval that ends *before* j begins. We define $p(j) = 0$ if no request $i < j$ is disjoint from j .

Notation and Simplifying Assumptions

Introducing Notation and Terminology

- Suppose that the requests are sorted with respect to finish times: $f_1 \leq f_2 \leq \dots \leq f_n$.
- We'll say a request i comes before a request j if $i < j$. This will be the natural left-to-right order in which we'll consider intervals.
- We define $p(j)$, for an interval j , to be the largest index $i < j$ such that intervals i and j are disjoint, i.e., i is the leftmost interval that ends *before* j begins. We define $p(j) = 0$ if no request $i < j$ is disjoint from j .

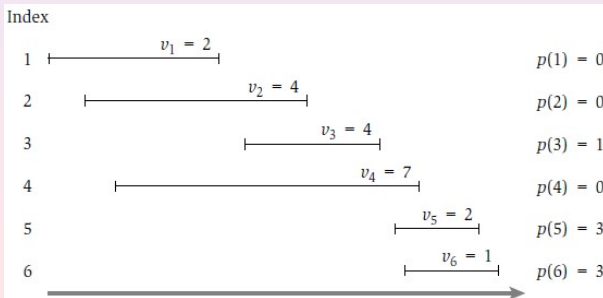


Figure: An Instance of Weighted Interval Scheduling with the function $p(j)$ defined for each interval.

Understanding The Optimal Solution

Identifying The Optimal Substructure Property

- Given an instance of the Weighted Interval Scheduling Problem, let's consider an optimal solution $\mathcal{O}$, ignoring for now that we have no idea what it is.

Understanding The Optimal Solution

Identifying The Optimal Substructure Property

- Given an instance of the Weighted Interval Scheduling Problem, let's consider an optimal solution $\mathcal{O}$, ignoring for now that we have no idea what it is.
- Here's something completely obvious that we can say about $\mathcal{O}$: either interval n (the last one) belongs to $\mathcal{O}$, or it doesn't.

Understanding The Optimal Solution

Identifying The Optimal Substructure Property

- Given an instance of the Weighted Interval Scheduling Problem, let's consider an optimal solution $\mathcal{O}$, ignoring for now that we have no idea what it is.
- Here's something completely obvious that we can say about $\mathcal{O}$: either interval n (the last one) belongs to $\mathcal{O}$, or it doesn't.
- Suppose we explore both sides of this dichotomy a little further.

Understanding The Optimal Solution

Identifying The Optimal Substructure Property

- Given an instance of the Weighted Interval Scheduling Problem, let's consider an optimal solution $\mathcal{O}$, ignoring for now that we have no idea what it is.
- Here's something completely obvious that we can say about $\mathcal{O}$: either interval n (the last one) belongs to $\mathcal{O}$, or it doesn't.
- Suppose we explore both sides of this dichotomy a little further.
- If $n \in \mathcal{O}$, then clearly no interval indexed strictly between $p(n)$ and n can belong to $\mathcal{O}$, because by the definition of $p(n)$, we know that intervals $p(n) + 1, p(n) + 2, \dots, n - 1$ all overlap interval n .

Understanding The Optimal Solution

Identifying The Optimal Substructure Property

- Given an instance of the Weighted Interval Scheduling Problem, let's consider an optimal solution $\mathcal{O}$, ignoring for now that we have no idea what it is.
- Here's something completely obvious that we can say about $\mathcal{O}$: either interval n (the last one) belongs to $\mathcal{O}$, or it doesn't.
- Suppose we explore both sides of this dichotomy a little further.
- If $n \in \mathcal{O}$, then clearly no interval indexed strictly between $p(n)$ and n can belong to $\mathcal{O}$, because by the definition of $p(n)$, we know that intervals $p(n) + 1, p(n) + 2, \dots, n - 1$ all overlap interval n .
- Moreover, if $n \in \mathcal{O}$, then $\mathcal{O}$ must include an *optimal* solution to the problem consisting of requests $\{1, \dots, p(n)\}$, since otherwise, we could replace $\mathcal{O}$'s choice of requests from $\{1, \dots, p(n)\}$ with a better one, with no danger of overlapping request n .

Understanding The Optimal Solution

Identifying The Optimal Substructure Property

- Given an instance of the Weighted Interval Scheduling Problem, let's consider an optimal solution $\mathcal{O}$, ignoring for now that we have no idea what it is.
- Here's something completely obvious that we can say about $\mathcal{O}$: either interval n (the last one) belongs to $\mathcal{O}$, or it doesn't.
- Suppose we explore both sides of this dichotomy a little further.
- If $n \in \mathcal{O}$, then clearly no interval indexed strictly between $p(n)$ and n can belong to $\mathcal{O}$, because by the definition of $p(n)$, we know that intervals $p(n) + 1, p(n) + 2, \dots, n - 1$ all overlap interval n .
- Moreover, if $n \in \mathcal{O}$, then $\mathcal{O}$ must include an *optimal* solution to the problem consisting of requests $\{1, \dots, p(n)\}$, since otherwise, we could replace $\mathcal{O}$'s choice of requests from $\{1, \dots, p(n)\}$ with a better one, with no danger of overlapping request n .
- On the other hand, if $n \notin \mathcal{O}$, then $\mathcal{O}$ is simply equal to the optimal solution to the problem consisting of requests $\{1, 2, \dots, n - 1\}$.

Understanding The Optimal Solution

Identifying The Optimal Substructure Property

- Given an instance of the Weighted Interval Scheduling Problem, let's consider an optimal solution $\mathcal{O}$, ignoring for now that we have no idea what it is.
- Here's something completely obvious that we can say about $\mathcal{O}$: either interval n (the last one) belongs to $\mathcal{O}$, or it doesn't.
- Suppose we explore both sides of this dichotomy a little further.
- If $n \in \mathcal{O}$, then clearly no interval indexed strictly between $p(n)$ and n can belong to $\mathcal{O}$, because by the definition of $p(n)$, we know that intervals $p(n) + 1, p(n) + 2, \dots, n - 1$ all overlap interval n .
- Moreover, if $n \in \mathcal{O}$, then $\mathcal{O}$ must include an *optimal* solution to the problem consisting of requests $\{1, \dots, p(n)\}$, since otherwise, we could replace $\mathcal{O}$'s choice of requests from $\{1, \dots, p(n)\}$ with a better one, with no danger of overlapping request n .
- On the other hand, if $n \notin \mathcal{O}$, then $\mathcal{O}$ is simply equal to the optimal solution to the problem consisting of requests $\{1, 2, \dots, n - 1\}$.
- This is by completely analogous reasoning: we're assuming that $\mathcal{O}$ does not include request n ; so if it does not choose the optimal set of requests from $\{1, 2, \dots, n - 1\}$, we could replace it with a better one.

Crucial Components of a Dynamic Program

Defining Subproblems and a Recurrence Linking Them

- Our observation suggests that finding an optimal solution on intervals $\{1, 2, \dots, n\}$ involves looking at the optimal solutions of smaller problems of the form $\{1, 2, \dots, j\}$.

Crucial Components of a Dynamic Program

Defining Subproblems and a Recurrence Linking Them

- Our observation suggests that finding an optimal solution on intervals $\{1, 2, \dots, n\}$ involves looking at the optimal solutions of smaller problems of the form $\{1, 2, \dots, j\}$.
- Thus, for any value of j between 1 and n , let $\mathcal{O}_j$ denote an optimal solution to the problem consisting of requests $\{1, \dots, j\}$, and let $OPT(j)$ denote the value of this solution.

Crucial Components of a Dynamic Program

Defining Subproblems and a Recurrence Linking Them

- Our observation suggests that finding an optimal solution on intervals $\{1, 2, \dots, n\}$ involves looking at the optimal solutions of smaller problems of the form $\{1, 2, \dots, j\}$.
- Thus, for any value of j between 1 and n , let $\mathcal{O}_j$ denote an optimal solution to the problem consisting of requests $\{1, \dots, j\}$, and let $OPT(j)$ denote the value of this solution.
- We define $OPT(0) = 0$, as the optimum over an empty set of intervals.

Crucial Components of a Dynamic Program

Defining Subproblems and a Recurrence Linking Them

- Our observation suggests that finding an optimal solution on intervals $\{1, 2, \dots, n\}$ involves looking at the optimal solutions of smaller problems of the form $\{1, 2, \dots, j\}$.
- Thus, for any value of j between 1 and n , let $\mathcal{O}_j$ denote an optimal solution to the problem consisting of requests $\{1, \dots, j\}$, and let $OPT(j)$ denote the value of this solution.
- We define $OPT(0) = 0$, as the optimum over an empty set of intervals.
- The optimal solution we're seeking is precisely $\mathcal{O}_n$, with value $OPT(n)$.

Crucial Components of a Dynamic Program

Defining Subproblems and a Recurrence Linking Them

- Our observation suggests that finding an optimal solution on intervals $\{1, 2, \dots, n\}$ involves looking at the optimal solutions of smaller problems of the form $\{1, 2, \dots, j\}$.
- Thus, for any value of j between 1 and n , let $\mathcal{O}_j$ denote an optimal solution to the problem consisting of requests $\{1, \dots, j\}$, and let $OPT(j)$ denote the value of this solution.
- We define $OPT(0) = 0$, as the optimum over an empty set of intervals.
- The optimal solution we're seeking is precisely $\mathcal{O}_n$, with value $OPT(n)$.
- For an optimal solution $\mathcal{O}_j$, our reasoning says that either $j \in \mathcal{O}_j$, in which case $OPT(j) = v_j + OPT(p(j))$, or $j \notin \mathcal{O}_j$, in which case $OPT(j) = OPT(j-1)$.

Crucial Components of a Dynamic Program

Defining Subproblems and a Recurrence Linking Them

- Our observation suggests that finding an optimal solution on intervals $\{1, 2, \dots, n\}$ involves looking at the optimal solutions of smaller problems of the form $\{1, 2, \dots, j\}$.
- Thus, for any value of j between 1 and n , let $\mathcal{O}_j$ denote an optimal solution to the problem consisting of requests $\{1, \dots, j\}$, and let $OPT(j)$ denote the value of this solution.
- We define $OPT(0) = 0$, as the optimum over an empty set of intervals.
- The optimal solution we're seeking is precisely $\mathcal{O}_n$, with value $OPT(n)$.
- For an optimal solution $\mathcal{O}_j$, our reasoning says that either $j \in \mathcal{O}_j$, in which case $OPT(j) = v_j + OPT(p(j))$, or $j \notin \mathcal{O}_j$, in which case $OPT(j) = OPT(j-1)$.
- Since these are precisely the two possible choices ($j \in \mathcal{O}_j$ or $j \notin \mathcal{O}_j$), we can further say that

$$OPT(j) = \max(v_j + OPT(p(j)), OPT(j-1)).$$

Crucial Components of a Dynamic Program

Defining Subproblems and a Recurrence Linking Them

- Our observation suggests that finding an optimal solution on intervals $\{1, 2, \dots, n\}$ involves looking at the optimal solutions of smaller problems of the form $\{1, 2, \dots, j\}$.
- Thus, for any value of j between 1 and n , let $\mathcal{O}_j$ denote an optimal solution to the problem consisting of requests $\{1, \dots, j\}$, and let $OPT(j)$ denote the value of this solution.
- We define $OPT(0) = 0$, as the optimum over an empty set of intervals.
- The optimal solution we're seeking is precisely $\mathcal{O}_n$, with value $OPT(n)$.
- For an optimal solution $\mathcal{O}_j$, our reasoning says that either $j \in \mathcal{O}_j$, in which case $OPT(j) = v_j + OPT(p(j))$, or $j \notin \mathcal{O}_j$, in which case $OPT(j) = OPT(j-1)$.
- Since these are precisely the two possible choices ($j \in \mathcal{O}_j$ or $j \notin \mathcal{O}_j$), we can further say that

$$OPT(j) = \max(v_j + OPT(p(j)), OPT(j-1)).$$

- How do we decide whether j belongs to an optimal solution $\mathcal{O}_j$?

Crucial Components of a Dynamic Program

Defining Subproblems and a Recurrence Linking Them

- Our observation suggests that finding an optimal solution on intervals $\{1, 2, \dots, n\}$ involves looking at the optimal solutions of smaller problems of the form $\{1, 2, \dots, j\}$.
- Thus, for any value of j between 1 and n , let $\mathcal{O}_j$ denote an optimal solution to the problem consisting of requests $\{1, \dots, j\}$, and let $OPT(j)$ denote the value of this solution.
- We define $OPT(0) = 0$, as the optimum over an empty set of intervals.
- The optimal solution we're seeking is precisely $\mathcal{O}_n$, with value $OPT(n)$.
- For an optimal solution $\mathcal{O}_j$, our reasoning says that either $j \in \mathcal{O}_j$, in which case $OPT(j) = v_j + OPT(p(j))$, or $j \notin \mathcal{O}_j$, in which case $OPT(j) = OPT(j-1)$.
- Since these are precisely the two possible choices ($j \in \mathcal{O}_j$ or $j \notin \mathcal{O}_j$), we can further say that

$$OPT(j) = \max(v_j + OPT(p(j)), OPT(j-1)).$$

- How do we decide whether j belongs to an optimal solution $\mathcal{O}_j$?
- It belongs to an optimal solution if and only if the first of the options above is at least as good as the second; in other words, Request j belongs to an optimal solution on the set $\{1, 2, \dots, j\}$ if and only if $v_j + OPT(p(j)) \geq OPT(j-1)$.

Obtaining An Algorithm for the Problem

A Recursive Implementation

- The facts we obtained so far form the crucial component on which a dynamic programming solution is based: a recurrence equation that expresses the optimal solution (or its value) in terms of the optimal solutions to smaller subproblems.

Obtaining An Algorithm for the Problem

A Recursive Implementation

- The facts we obtained so far form the crucial component on which a dynamic programming solution is based: a recurrence equation that expresses the optimal solution (or its value) in terms of the optimal solutions to smaller subproblems.
- Despite the simple reasoning that led to this point, our recurrence directly gives us a recursive algorithm to compute $OPT(n)$, assuming that we have already sorted the requests by finishing time and computed the values of $p(j)$ for each j .

Obtaining An Algorithm for the Problem

A Recursive Implementation

- The facts we obtained so far form the crucial component on which a dynamic programming solution is based: a recurrence equation that expresses the optimal solution (or its value) in terms of the optimal solutions to smaller subproblems.
- Despite the simple reasoning that led to this point, our recurrence directly gives us a recursive algorithm to compute $OPT(n)$, assuming that we have already sorted the requests by finishing time and computed the values of $p(j)$ for each j .

```
Compute-Opt(j)
  If  $j=0$  then
    Return 0
  Else
    Return  $\max(v_j + \text{Compute-Opt}(p(j)), \text{Compute-Opt}(j-1))$ 
  Endif
```

Figure: If we implement this algorithm as just written, it would take exponential time to run in the worst case.

The Tree of Function Calls

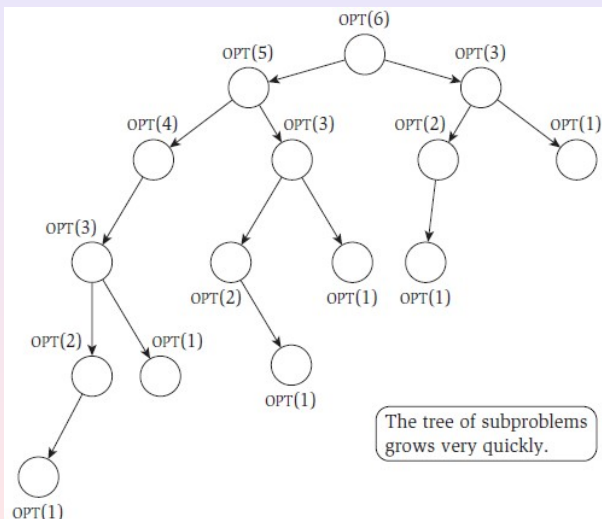


Figure: The tree of subproblems called by Compute-OPT on our example problem instance.

Number of Recursive Calls May Be As High As Fibonacci Numbers



Figure: What can you say about the number of recursive calls in this instance?

Number of Recursive Calls May Be As High As Fibonacci Numbers

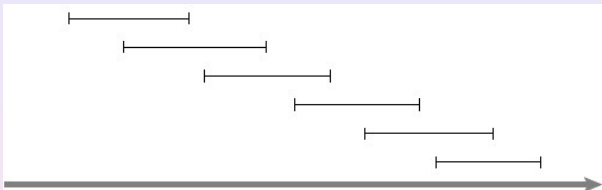


Figure: What can you say about the number of recursive calls in this instance?

Analyzing The Worst-Case

- The instance in the figure is the worst-case example, where $p(j) = j - 2$ for each $j = 2, 3, 4, \dots, n$.

Number of Recursive Calls May Be As High As Fibonacci Numbers

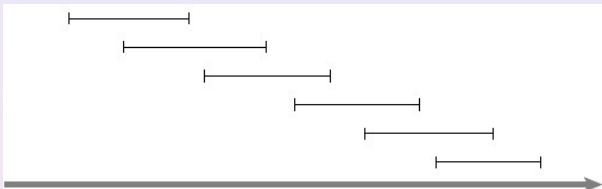


Figure: What can you say about the number of recursive calls in this instance?

Analyzing The Worst-Case

- The instance in the figure is the worst-case example, where $p(j) = j - 2$ for each $j = 2, 3, 4, \dots, n$.
- $\text{Compute-Opt}(j)$ generates separate recursive calls on problems of sizes $j - 1$ and $j - 2$.

Number of Recursive Calls May Be As High As Fibonacci Numbers

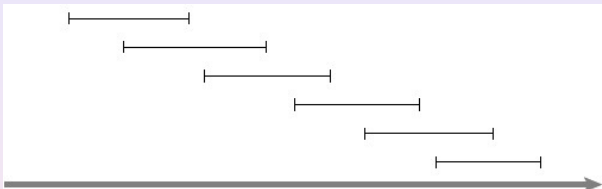


Figure: What can you say about the number of recursive calls in this instance?

Analyzing The Worst-Case

- The instance in the figure is the worst-case example, where $p(j) = j - 2$ for each $j = 2, 3, 4, \dots, n$.
- $\text{Compute-Opt}(j)$ generates separate recursive calls on problems of sizes $j - 1$ and $j - 2$.
- Notice that the total number of calls made to Compute-Opt on this instance will grow like the Fibonacci numbers, which increase exponentially.

Number of Recursive Calls May Be As High As Fibonacci Numbers

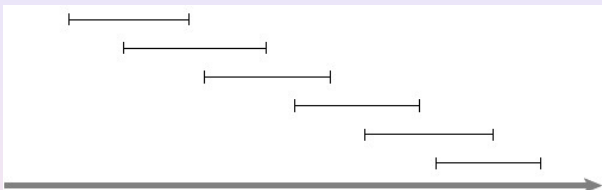


Figure: What can you say about the number of recursive calls in this instance?

Analyzing The Worst-Case

- The instance in the figure is the worst-case example, where $p(j) = j - 2$ for each $j = 2, 3, 4, \dots, n$.
- $\text{Compute-Opt}(j)$ generates separate recursive calls on problems of sizes $j - 1$ and $j - 2$.
- Notice that the total number of calls made to Compute-Opt on this instance will grow like the Fibonacci numbers, which increase exponentially.
- Thus we have not achieved a polynomial-time solution.

A Technique To Avoid Redundant Computation

Memoization

- A fundamental observation, which forms another crucial component of a dynamic programming solution, is that our recursive algorithm `ComputeOpt` is really only solving $n + 1$ different subproblems: `Compute-Opt(0)`, `Compute-Opt(1)`, $\dots$, `Compute-Opt(n)`.

A Technique To Avoid Redundant Computation

Memoization

- A fundamental observation, which forms another crucial component of a dynamic programming solution, is that our recursive algorithm `ComputeOpt` is really only solving $n + 1$ different subproblems: `Compute-Opt(0)`, `Compute-Opt(1)`, ..., `Compute-Opt(n)`.
- The fact that it runs in exponential time as written is simply due to the spectacular redundancy in the number of times it issues each of these calls.

A Technique To Avoid Redundant Computation

Memoization

- A fundamental observation, which forms another crucial component of a dynamic programming solution, is that our recursive algorithm `ComputeOpt` is really only solving $n + 1$ different subproblems: `Compute-Opt(0)`, `Compute-Opt(1)`, ..., `Compute-Opt(n)`.
- The fact that it runs in exponential time as written is simply due to the spectacular redundancy in the number of times it issues each of these calls.
- How could we eliminate all this redundancy?

A Technique To Avoid Redundant Computation

Memoization

- A fundamental observation, which forms another crucial component of a dynamic programming solution, is that our recursive algorithm `ComputeOpt` is really only solving $n + 1$ different subproblems: `Compute-Opt(0)`, `Compute-Opt(1)`, $\dots$, `Compute-Opt(n)`.
- The fact that it runs in exponential time as written is simply due to the spectacular redundancy in the number of times it issues each of these calls.
- How could we eliminate all this redundancy?
- We could store the value of `Compute-Opt` in a globally accessible place the first time we compute it and then simply use this precomputed value in place of all future recursive calls.

A Technique To Avoid Redundant Computation

Memoization

- A fundamental observation, which forms another crucial component of a dynamic programming solution, is that our recursive algorithm `ComputeOpt` is really only solving $n + 1$ different subproblems: `Compute-Opt(0)`, `Compute-Opt(1)`, ..., `Compute-Opt(n)`.
- The fact that it runs in exponential time as written is simply due to the spectacular redundancy in the number of times it issues each of these calls.
- How could we eliminate all this redundancy?
- We could store the value of `Compute-Opt` in a globally accessible place the first time we compute it and then simply use this precomputed value in place of all future recursive calls.
- This technique of saving values that have already been computed is referred to as **memoization**.

A Technique To Avoid Redundant Computation

Memoization

- A fundamental observation, which forms another crucial component of a dynamic programming solution, is that our recursive algorithm `ComputeOpt` is really only solving $n + 1$ different subproblems: `Compute-Opt(0)`, `Compute-Opt(1)`, ..., `Compute-Opt(n)`.
- The fact that it runs in exponential time as written is simply due to the spectacular redundancy in the number of times it issues each of these calls.
- How could we eliminate all this redundancy?
- We could store the value of `Compute-Opt` in a globally accessible place the first time we compute it and then simply use this precomputed value in place of all future recursive calls.
- This technique of saving values that have already been computed is referred to as **memoization**.
- We implement this strategy in the more "intelligent" procedure `MCompute-Opt`.

A Technique To Avoid Redundant Computation

Memoization

- A fundamental observation, which forms another crucial component of a dynamic programming solution, is that our recursive algorithm `ComputeOpt` is really only solving $n + 1$ different subproblems: `Compute-Opt(0)`, `Compute-Opt(1)`, ..., `Compute-Opt(n)`.
- The fact that it runs in exponential time as written is simply due to the spectacular redundancy in the number of times it issues each of these calls.
- How could we eliminate all this redundancy?
- We could store the value of `Compute-Opt` in a globally accessible place the first time we compute it and then simply use this precomputed value in place of all future recursive calls.
- This technique of saving values that have already been computed is referred to as **memoization**.
- We implement this strategy in the more "intelligent" procedure `MCompute-Opt`.
- This procedure will make use of an array $M[0 \dots n]$; $M[j]$ will start with the value "empty", but will hold the value of `Compute-Opt(j)` as soon as it is first determined. To determine $OPT(n)$, we invoke `M-ComputeOpt(n)`.

Memoized Implementation

```
M-Compute-Opt(j)
  If j = 0 then
    Return 0
  Else if M[j] is not empty then
    Return M[j]
  Else
    Define M[j] = max( $v_j + \text{M-Compute-Opt}(p(j))$ , M-Compute-Opt(j - 1))
    Return M[j]
  Endif
```

Figure: An $O(n)$ implementation.

Computing an Optimal Solution

Discussing a Naïve and a Novel Strategy

- So far we have simply computed the value of an optimal solution; presumably we want a full optimal set of intervals as well.

Computing an Optimal Solution

Discussing a Naïve and a Novel Strategy

- So far we have simply computed the value of an optimal solution; presumably we want a full optimal set of intervals as well.
- It would be easy to extend M-Compute-Opt so as to keep track of an optimal solution in addition to its value: we could maintain an additional array S so that $S[i]$ contains an optimal set of intervals among $\{1, 2, \dots, i\}$.

Computing an Optimal Solution

Discussing a Naïve and a Novel Strategy

- So far we have simply computed the value of an optimal solution; presumably we want a full optimal set of intervals as well.
- It would be easy to extend M-Compute-Opt so as to keep track of an optimal solution in addition to its value: we could maintain an additional array S so that $S[i]$ contains an optimal set of intervals among $\{1, 2, \dots, i\}$.
- Naïvely enhancing the code to maintain the solutions in the array S , however, would blow up the running time by an additional factor of $O(n)$: while a position in the M array can be updated in $O(1)$ time, writing down a set in the S array takes $O(n)$ time.

Computing an Optimal Solution

Discussing a Naïve and a Novel Strategy

- So far we have simply computed the value of an optimal solution; presumably we want a full optimal set of intervals as well.
- It would be easy to extend M-Compute-Opt so as to keep track of an optimal solution in addition to its value: we could maintain an additional array S so that $S[i]$ contains an optimal set of intervals among $\{1, 2, \dots, i\}$.
- Naïvely enhancing the code to maintain the solutions in the array S , however, would blow up the running time by an additional factor of $O(n)$: while a position in the M array can be updated in $O(1)$ time, writing down a set in the S array takes $O(n)$ time.
- We can avoid this $O(n)$ blow-up by not explicitly maintaining S , but rather by recovering an optimal solution from values saved in the array M after the optimum value has been computed.

Computing an Optimal Solution

Discussing a Naïve and a Novel Strategy

- So far we have simply computed the value of an optimal solution; presumably we want a full optimal set of intervals as well.
- It would be easy to extend M-Compute-Opt so as to keep track of an optimal solution in addition to its value: we could maintain an additional array S so that $S[i]$ contains an optimal set of intervals among $\{1, 2, \dots, i\}$.
- Naïvely enhancing the code to maintain the solutions in the array S , however, would blow up the running time by an additional factor of $O(n)$: while a position in the M array can be updated in $O(1)$ time, writing down a set in the S array takes $O(n)$ time.
- We can avoid this $O(n)$ blow-up by not explicitly maintaining S , but rather by recovering an optimal solution from values saved in the array M after the optimum value has been computed.
- We know that j belongs to an optimal solution for the set of intervals $\{1, \dots, j\}$ if and only if $v_j + \text{OPT}(p(j)) \geq \text{OPT}(j - 1)$.

Computing an Optimal Solution

Discussing a Naïve and a Novel Strategy

- So far we have simply computed the value of an optimal solution; presumably we want a full optimal set of intervals as well.
- It would be easy to extend M-Compute-Opt so as to keep track of an optimal solution in addition to its value: we could maintain an additional array S so that $S[i]$ contains an optimal set of intervals among $\{1, 2, \dots, i\}$.
- Naïvely enhancing the code to maintain the solutions in the array S , however, would blow up the running time by an additional factor of $O(n)$: while a position in the M array can be updated in $O(1)$ time, writing down a set in the S array takes $O(n)$ time.
- We can avoid this $O(n)$ blow-up by not explicitly maintaining S , but rather by recovering an optimal solution from values saved in the array M after the optimum value has been computed.
- We know that j belongs to an optimal solution for the set of intervals $\{1, \dots, j\}$ if and only if $v_j + \text{OPT}(p(j)) \geq \text{OPT}(j - 1)$.
- Using this observation, we get write a simple procedure, which "traces back" through the array M to find the set of intervals in an optimal solution.

Pseudocode For Computing Optimal Solution

```
Find-Solution( $j$ )  
  If  $j = 0$  then  
    Output nothing  
  Else  
    If  $v_j + M[p(j)] \geq M[j - 1]$  then  
      Output  $j$  together with the result of Find-Solution( $p(j)$ )  
    Else  
      Output the result of Find-Solution( $j - 1$ )  
    Endif  
  Endif
```

Figure: Given the array M of the optimal values of the sub-problems, Find-Solution returns an optimal solution in $O(n)$ time.

Designing Another Efficient Algorithm

Iterative Implementation

- The key to the efficient memoized algorithm is really the array M .

Designing Another Efficient Algorithm

Iterative Implementation

- The key to the efficient memoized algorithm is really the array M .
- It encodes the notion that we're using the value of optimal solutions to the sub-problems on intervals $\{1, 2, \dots, j\}$ for each j , and it uses $OPT(j) = \max(v_j + OPT(p(j)), OPT(j-1))$ to define the value of $M[j]$ based on values that come earlier in the array.

Designing Another Efficient Algorithm

Iterative Implementation

- The key to the efficient memoized algorithm is really the array M .
- It encodes the notion that we're using the value of optimal solutions to the sub-problems on intervals $\{1, 2, \dots, j\}$ for each j , and it uses $OPT(j) = \max(v_j + OPT(p(j)), OPT(j-1))$ to define the value of $M[j]$ based on values that come earlier in the array.
- Once we have the array M , the problem is solved: $M[n]$ contains the value of the optimal solution on the full instance, and Find-Solution can be used to trace back through M efficiently and return an optimal solution itself.

Designing Another Efficient Algorithm

Iterative Implementation

- The key to the efficient memoized algorithm is really the array M .
- It encodes the notion that we're using the value of optimal solutions to the sub-problems on intervals $\{1, 2, \dots, j\}$ for each j , and it uses $OPT(j) = \max(v_j + OPT(p(j)), OPT(j-1))$ to define the value of $M[j]$ based on values that come earlier in the array.
- Once we have the array M , the problem is solved: $M[n]$ contains the value of the optimal solution on the full instance, and Find-Solution can be used to trace back through M efficiently and return an optimal solution itself.
- The point to realize, then, is that we can directly compute the entries in M by an iterative algorithm, rather than using memoized recursion.

Designing Another Efficient Algorithm

Iterative Implementation

- The key to the efficient memoized algorithm is really the array M .
- It encodes the notion that we're using the value of optimal solutions to the sub-problems on intervals $\{1, 2, \dots, j\}$ for each j , and it uses $OPT(j) = \max(v_j + OPT(p(j)), OPT(j-1))$ to define the value of $M[j]$ based on values that come earlier in the array.
- Once we have the array M , the problem is solved: $M[n]$ contains the value of the optimal solution on the full instance, and Find-Solution can be used to trace back through M efficiently and return an optimal solution itself.
- The point to realize, then, is that we can directly compute the entries in M by an iterative algorithm, rather than using memoized recursion.
- We just start with $M[0] = 0$ and keep incrementing j ; each time we need to determine a value $M[j]$, the answer is provided by $OPT(j) = \max(v_j + OPT(p(j)), OPT(j-1))$.

Pseudocode For Iterative Implementation

```
Iterative-Compute-Opt
```

```
     $M[0] = 0$ 
```

```
    For  $j = 1, 2, \dots, n$ 
```

```
         $M[j] = \max(v_j + M[p(j)], M[j - 1])$ 
```

```
    Endfor
```

A Rough Template of Designing Dynamic Programming Algorithms

General Principles

- Developing an algorithm based on dynamic programming, one needs a collection of sub-problems derived from the original problem that satisfies a few basic properties:
 - 1 There are only a polynomial number of sub-problems.

A Rough Template of Designing Dynamic Programming Algorithms

General Principles

- Developing an algorithm based on dynamic programming, one needs a collection of sub-problems derived from the original problem that satisfies a few basic properties:
 - 1 There are only a polynomial number of sub-problems.
 - 2 The solution to the original problem can be easily computed from the solutions to the sub-problems. For example, the original problem may actually be one of the sub-problems.

A Rough Template of Designing Dynamic Programming Algorithms

General Principles

- Developing an algorithm based on dynamic programming, one needs a collection of sub-problems derived from the original problem that satisfies a few basic properties:
 - 1 There are only a polynomial number of sub-problems.
 - 2 The solution to the original problem can be easily computed from the solutions to the sub-problems. For example, the original problem may actually be one of the sub-problems.
 - 3 There is a natural ordering on sub-problems from "smallest" to "largest", together with an easy-to-compute recurrence that allows one to determine the solution to a sub-problem from the solutions to some number of smaller sub-problems.

A Rough Template of Designing Dynamic Programming Algorithms

General Principles

- Developing an algorithm based on dynamic programming, one needs a collection of sub-problems derived from the original problem that satisfies a few basic properties:
 - 1 There are only a polynomial number of sub-problems.
 - 2 The solution to the original problem can be easily computed from the solutions to the sub-problems. For example, the original problem may actually be one of the sub-problems.
 - 3 There is a natural ordering on sub-problems from "smallest" to "largest", together with an easy-to-compute recurrence that allows one to determine the solution to a sub-problem from the solutions to some number of smaller sub-problems.
- It is sometimes easier to start the process of designing such an algorithm by formulating a set of sub-problems that looks natural, and then figuring out a recurrence that links them together; but often (as happened in the case of weighted interval scheduling), it can be useful to first define a recurrence by reasoning about the structure of an optimal solution, and then determine which sub-problems will be necessary to unwind the recurrence.

A Rough Template of Designing Dynamic Programming Algorithms

General Principles

- Developing an algorithm based on dynamic programming, one needs a collection of sub-problems derived from the original problem that satisfies a few basic properties:
 - 1 There are only a polynomial number of sub-problems.
 - 2 The solution to the original problem can be easily computed from the solutions to the sub-problems. For example, the original problem may actually be one of the sub-problems.
 - 3 There is a natural ordering on sub-problems from "smallest" to "largest", together with an easy-to-compute recurrence that allows one to determine the solution to a sub-problem from the solutions to some number of smaller sub-problems.
- It is sometimes easier to start the process of designing such an algorithm by formulating a set of sub-problems that looks natural, and then figuring out a recurrence that links them together; but often (as happened in the case of weighted interval scheduling), it can be useful to first define a recurrence by reasoning about the structure of an optimal solution, and then determine which sub-problems will be necessary to unwind the recurrence.
- This chicken-and-egg relationship between sub-problems and recurrences is a subtle issue underlying dynamic programming.

A Rough Template of Designing Dynamic Programming Algorithms

General Principles

- Developing an algorithm based on dynamic programming, one needs a collection of sub-problems derived from the original problem that satisfies a few basic properties:
 - 1 There are only a polynomial number of sub-problems.
 - 2 The solution to the original problem can be easily computed from the solutions to the sub-problems. For example, the original problem may actually be one of the sub-problems.
 - 3 There is a natural ordering on sub-problems from "smallest" to "largest", together with an easy-to-compute recurrence that allows one to determine the solution to a sub-problem from the solutions to some number of smaller sub-problems.
- It is sometimes easier to start the process of designing such an algorithm by formulating a set of sub-problems that looks natural, and then figuring out a recurrence that links them together; but often (as happened in the case of weighted interval scheduling), it can be useful to first define a recurrence by reasoning about the structure of an optimal solution, and then determine which sub-problems will be necessary to unwind the recurrence.
- This chicken-and-egg relationship between sub-problems and recurrences is a subtle issue underlying dynamic programming.
- It's never clear that a collection of sub-problems will be useful until one finds a recurrence linking them together; but it can be difficult to think about recurrences in the absence of the "smaller" subproblems that they build on.

Rod Cutting Problem

Problem Definition

- Consider a company that buys long steel rods and cuts them into shorter rods, which it then sells. Each cut is free.

Rod Cutting Problem

Problem Definition

- Consider a company that buys long steel rods and cuts them into shorter rods, which it then sells. Each cut is free.
- The management of the company wants to know the best way to cut up the rods.

Rod Cutting Problem

Problem Definition

- Consider a company that buys long steel rods and cuts them into shorter rods, which it then sells. Each cut is free.
- The management of the company wants to know the best way to cut up the rods.
- The company has a table giving, for $i = 1, 2, \dots$, the price p_i in dollars that they charge for a rod of length i inches. The length of each rod in inches is always an integer.

Rod Cutting Problem

Problem Definition

- Consider a company that buys long steel rods and cuts them into shorter rods, which it then sells. Each cut is free.
- The management of the company wants to know the best way to cut up the rods.
- The company has a table giving, for $i = 1, 2, \dots$, the price p_i in dollars that they charge for a rod of length i inches. The length of each rod in inches is always an integer.
- **Rod-cutting problem:** Given a rod of length n inches and a table of prices p_i for $i = 1, 2, \dots, n$, determine the maximum revenue r_n obtainable by cutting up the rod and selling the pieces.

Rod Cutting Problem

Problem Definition

- Consider a company that buys long steel rods and cuts them into shorter rods, which it then sells. Each cut is free.
- The management of the company wants to know the best way to cut up the rods.
- The company has a table giving, for $i = 1, 2, \dots$, the price p_i in dollars that they charge for a rod of length i inches. The length of each rod in inches is always an integer.
- **Rod-cutting problem:** Given a rod of length n inches and a table of prices p_i for $i = 1, 2, \dots, n$, determine the maximum revenue r_n obtainable by cutting up the rod and selling the pieces.
- If the price p_n for a rod of length n is large enough, an optimal solution might require no cutting at all.

Illustrating The Problem Through An Instance

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

Figure: A sample price list.

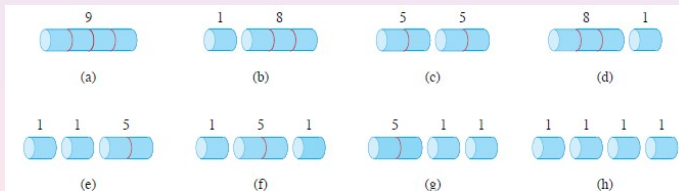


Figure: Company can cut up a rod of length n in 2^{n-1} different ways, since they have an independent option of cutting, or not cutting, at distance i inches from the left end, for $i = 1, 2, \dots, n-1$. The 8 possible ways of cutting up a rod of length 4 is given in the figure. Above each piece is the value of that piece, according to the sample price chart. The optimal strategy is cutting the rod into two pieces of length 2, which has total value 10.

Obtaining A Recurrence for the Value of Optimal Solution

Explanation

- We denote a decomposition into pieces using ordinary additive notation, so that $7 = 2 + 2 + 3$ indicates that a rod of length 7 is cut into three pieces: two of length 2 and one of length 3.

Obtaining A Recurrence for the Value of Optimal Solution

Explanation

- We denote a decomposition into pieces using ordinary additive notation, so that $7 = 2 + 2 + 3$ indicates that a rod of length 7 is cut into three pieces: two of length 2 and one of length 3.
- If an optimal solution cuts the rod into k pieces, for some $1 \leq k \leq n$, then an optimal decomposition $n = i_1 + i_2 + \dots + i_k$ of the rod into pieces of lengths $i_1, i_2, \dots, i_k$ provides maximum corresponding revenue $r_n = p_{i_1} + p_{i_2} + \dots + p_{i_k}$.

Obtaining A Recurrence for the Value of Optimal Solution

Explanation

- We denote a decomposition into pieces using ordinary additive notation, so that $7 = 2 + 2 + 3$ indicates that a rod of length 7 is cut into three pieces: two of length 2 and one of length 3.
- If an optimal solution cuts the rod into k pieces, for some $1 \leq k \leq n$, then an optimal decomposition $n = i_1 + i_2 + \dots + i_k$ of the rod into pieces of lengths $i_1, i_2, \dots, i_k$ provides maximum corresponding revenue $r_n = p_{i_1} + p_{i_2} + \dots + p_{i_k}$.
- Every decomposition of a length- n rod consists of a first piece of length i ($1 \leq i \leq n$) cut off the left-hand end, and then a right-hand remainder of length $n - i$, where the remainder may be further divided.

Obtaining A Recurrence for the Value of Optimal Solution

Explanation

- We denote a decomposition into pieces using ordinary additive notation, so that $7 = 2 + 2 + 3$ indicates that a rod of length 7 is cut into three pieces: two of length 2 and one of length 3.
- If an optimal solution cuts the rod into k pieces, for some $1 \leq k \leq n$, then an optimal decomposition $n = i_1 + i_2 + \dots + i_k$ of the rod into pieces of lengths $i_1, i_2, \dots, i_k$ provides maximum corresponding revenue $r_n = p_{i_1} + p_{i_2} + \dots + p_{i_k}$.
- Every decomposition of a length- n rod consists of a first piece of length i ($1 \leq i \leq n$) cut off the left-hand end, and then a right-hand remainder of length $n - i$, where the remainder may be further divided.
- Notice that in the solution with no cuts first piece has size $i = n$ and revenue p_n , and the remainder has size 0 with corresponding revenue $r_0 = 0$.

Obtaining A Recurrence for the Value of Optimal Solution

Explanation

- We denote a decomposition into pieces using ordinary additive notation, so that $7 = 2 + 2 + 3$ indicates that a rod of length 7 is cut into three pieces: two of length 2 and one of length 3.
- If an optimal solution cuts the rod into k pieces, for some $1 \leq k \leq n$, then an optimal decomposition $n = i_1 + i_2 + \dots + i_k$ of the rod into pieces of lengths $i_1, i_2, \dots, i_k$ provides maximum corresponding revenue $r_n = p_{i_1} + p_{i_2} + \dots + p_{i_k}$.
- Every decomposition of a length- n rod consists of a first piece of length i ($1 \leq i \leq n$) cut off the left-hand end, and then a right-hand remainder of length $n - i$, where the remainder may be further divided.
- Notice that in the solution with no cuts first piece has size $i = n$ and revenue p_n , and the remainder has size 0 with corresponding revenue $r_0 = 0$.
- We thus obtain the following equation that expresses the revenue of the optimal solution:

$$r_n = \max\{p_i + r_{n-i} : 1 \leq i \leq n\}.$$

Pseudocode for Iterative Implementation

BOTTOM-UP-CUT-ROD(p, n)

```
1  let  $r[0:n]$  be a new array           // will remember solution values in  $r$ 
2   $r[0] = 0$ 
3  for  $j = 1$  to  $n$                        // for increasing rod length  $j$ 
4       $q = -\infty$ 
5      for  $i = 1$  to  $j$                    //  $i$  is the position of the first cut
6           $q = \max\{q, p[i] + r[j - i]\}$ 
7       $r[j] = q$                          // remember the solution value for length  $j$ 
8  return  $r[n]$ 
```

Figure: A $\Theta(n^2)$ iterative dynamic programming algorithm for the Rod-Cutting problem.

Pseudocode For Computing an Optimal Solution

EXTENDED-BOTTOM-UP-CUT-ROD(p, n)

```

1  let  $r[0:n]$  and  $s[1:n]$  be new arrays
2   $r[0] = 0$ 
3  for  $j = 1$  to  $n$                                 // for increasing rod length  $j$ 
4       $q = -\infty$ 
5      for  $i = 1$  to  $j$                             //  $i$  is the position of the first cut
6          if  $q < p[i] + r[j - i]$ 
7               $q = p[i] + r[j - i]$ 
8               $s[j] = i$                             // best cut location so far for length  $j$ 
9       $r[j] = q$                                     // remember the solution value for length  $j$ 
10 return  $r$  and  $s$ 

```

PRINT-CUT-ROD-SOLUTION(p, n)

```

1  (r, s) = EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
2  while n > 0
3      print s[n]           // cut location for length n
4      n = n - s[n]         // length of the remainder of the rod

```


Subset Sum and Knapsack Problems

Problem Definitions

- We have a single machine that can process jobs, and we have a set of requests $\{1, 2, \dots, n\}$.

Subset Sum and Knapsack Problems

Problem Definitions

- We have a single machine that can process jobs, and we have a set of requests $\{1, 2, \dots, n\}$.
- We are only able to use this resource for the period between time 0 and time W , for some number W .

Subset Sum and Knapsack Problems

Problem Definitions

- We have a single machine that can process jobs, and we have a set of requests $\{1, 2, \dots, n\}$.
- We are only able to use this resource for the period between time 0 and time W , for some number W .
- Each request corresponds to a job that requires time w_i to process. If our goal is to process jobs so as to keep the machine as busy as possible up to the "cut-off" W , which jobs should we choose?

Subset Sum and Knapsack Problems

Problem Definitions

- We have a single machine that can process jobs, and we have a set of requests $\{1, 2, \dots, n\}$.
- We are only able to use this resource for the period between time 0 and time W , for some number W .
- Each request corresponds to a job that requires time w_i to process. If our goal is to process jobs so as to keep the machine as busy as possible up to the "cut-off" W , which jobs should we choose?
- More formally, we are given n items $\{1, \dots, n\}$, and each has a given nonnegative weight w_i (for $i = 1, \dots, n$). We are also given a bound W .

Subset Sum and Knapsack Problems

Problem Definitions

- We have a single machine that can process jobs, and we have a set of requests $\{1, 2, \dots, n\}$.
- We are only able to use this resource for the period between time 0 and time W , for some number W .
- Each request corresponds to a job that requires time w_i to process. If our goal is to process jobs so as to keep the machine as busy as possible up to the "cut-off" W , which jobs should we choose?
- More formally, we are given n items $\{1, \dots, n\}$, and each has a given nonnegative weight w_i (for $i = 1, \dots, n$). We are also given a bound W .
- We would like to select a subset S of the items so that $\sum_{i \in S} w_i \leq W$ and, subject to this restriction, $\sum_{i \in S} w_i$ is as large as possible. We will call this the **Subset Sum Problem**.

Subset Sum and Knapsack Problems

Problem Definitions

- We have a single machine that can process jobs, and we have a set of requests $\{1, 2, \dots, n\}$.
- We are only able to use this resource for the period between time 0 and time W , for some number W .
- Each request corresponds to a job that requires time w_i to process. If our goal is to process jobs so as to keep the machine as busy as possible up to the "cut-off" W , which jobs should we choose?
- More formally, we are given n items $\{1, \dots, n\}$, and each has a given nonnegative weight w_i (for $i = 1, \dots, n$). We are also given a bound W .
- We would like to select a subset S of the items so that $\sum_{i \in S} w_i \leq W$ and, subject to this restriction, $\sum_{i \in S} w_i$ is as large as possible. We will call this the **Subset Sum Problem**.
- This problem is a natural special case of a more general problem called the **Knapsack Problem**, where each request i has both a value v_i and a weight w_i .

Subset Sum and Knapsack Problems

Problem Definitions

- We have a single machine that can process jobs, and we have a set of requests $\{1, 2, \dots, n\}$.
- We are only able to use this resource for the period between time 0 and time W , for some number W .
- Each request corresponds to a job that requires time w_i to process. If our goal is to process jobs so as to keep the machine as busy as possible up to the "cut-off" W , which jobs should we choose?
- More formally, we are given n items $\{1, \dots, n\}$, and each has a given nonnegative weight w_i (for $i = 1, \dots, n$). We are also given a bound W .
- We would like to select a subset S of the items so that $\sum_{i \in S} w_i \leq W$ and, subject to this restriction, $\sum_{i \in S} w_i$ is as large as possible. We will call this the **Subset Sum Problem**.
- This problem is a natural special case of a more general problem called the **Knapsack Problem**, where each request i has both a value v_i and a weight w_i .
- The goal in this more general problem is to select a subset of maximum total value, subject to the restriction that its total weight not exceed W .
- We will use weight or time when referring to the quantities w_i and W .

Designing the Algorithm

A False Start

- One general strategy, which worked for us in the case of Weighted Interval Scheduling, is to consider sub-problems involving only the first i requests.

Designing the Algorithm

A False Start

- One general strategy, which worked for us in the case of Weighted Interval Scheduling, is to consider sub-problems involving only the first i requests.
- Let $OPT(i)$ denote the best possible solution using the subset of the requests $\{1, \dots, i\}$.

Designing the Algorithm

A False Start

- One general strategy, which worked for us in the case of Weighted Interval Scheduling, is to consider sub-problems involving only the first i requests.
- Let $OPT(i)$ denote the best possible solution using the subset of the requests $\{1, \dots, i\}$.
- The key to our method for the Weighted Interval Scheduling Problem was to concentrate on an optimal solution $\mathcal{O}$ to our problem and consider two cases, depending on whether or not the last request n is accepted or rejected by $\mathcal{O}$.

Designing the Algorithm

A False Start

- One general strategy, which worked for us in the case of Weighted Interval Scheduling, is to consider sub-problems involving only the first i requests.
- Let $OPT(i)$ denote the best possible solution using the subset of the requests $\{1, \dots, i\}$.
- The key to our method for the Weighted Interval Scheduling Problem was to concentrate on an optimal solution $\mathcal{O}$ to our problem and consider two cases, depending on whether or not the last request n is accepted or rejected by $\mathcal{O}$.
- Just as in that case, we have the first part, which follows immediately from the definition of $OPT(i)$: If $n \notin \mathcal{O}$, then $OPT(n) = OPT(n-1)$.

Designing the Algorithm

A False Start

- One general strategy, which worked for us in the case of Weighted Interval Scheduling, is to consider sub-problems involving only the first i requests.
- Let $OPT(i)$ denote the best possible solution using the subset of the requests $\{1, \dots, i\}$.
- The key to our method for the Weighted Interval Scheduling Problem was to concentrate on an optimal solution $\mathcal{O}$ to our problem and consider two cases, depending on whether or not the last request n is accepted or rejected by $\mathcal{O}$.
- Just as in that case, we have the first part, which follows immediately from the definition of $OPT(i)$: If $n \notin \mathcal{O}$, then $OPT(n) = OPT(n-1)$.
- Next we have to consider the case in which $n \in \mathcal{O}$. What we'd like here is a simple recursion, which tells us the best possible value we can get for solutions that contain the last request n .

Designing the Algorithm

A False Start

- One general strategy, which worked for us in the case of Weighted Interval Scheduling, is to consider sub-problems involving only the first i requests.
- Let $OPT(i)$ denote the best possible solution using the subset of the requests $\{1, \dots, i\}$.
- The key to our method for the Weighted Interval Scheduling Problem was to concentrate on an optimal solution $\mathcal{O}$ to our problem and consider two cases, depending on whether or not the last request n is accepted or rejected by $\mathcal{O}$.
- Just as in that case, we have the first part, which follows immediately from the definition of $OPT(i)$: If $n \notin \mathcal{O}$, then $OPT(n) = OPT(n-1)$.
- Next we have to consider the case in which $n \in \mathcal{O}$. What we'd like here is a simple recursion, which tells us the best possible value we can get for solutions that contain the last request n .
- For Weighted Interval Scheduling, we could simply delete each request that conflicted with request n . In the current problem, accepting request n does not immediately imply that we have to reject any other request.

Designing the Algorithm

A False Start

- One general strategy, which worked for us in the case of Weighted Interval Scheduling, is to consider sub-problems involving only the first i requests.
- Let $OPT(i)$ denote the best possible solution using the subset of the requests $\{1, \dots, i\}$.
- The key to our method for the Weighted Interval Scheduling Problem was to concentrate on an optimal solution $\mathcal{O}$ to our problem and consider two cases, depending on whether or not the last request n is accepted or rejected by $\mathcal{O}$.
- Just as in that case, we have the first part, which follows immediately from the definition of $OPT(i)$: If $n \notin \mathcal{O}$, then $OPT(n) = OPT(n-1)$.
- Next we have to consider the case in which $n \in \mathcal{O}$. What we'd like here is a simple recursion, which tells us the best possible value we can get for solutions that contain the last request n .
- For Weighted Interval Scheduling, we could simply delete each request that conflicted with request n . In the current problem, accepting request n does not immediately imply that we have to reject any other request.
- Instead, it means that for the subset of requests $S \subseteq \{1, 2, \dots, n-1\}$ that we will accept, we have less available weight left: a weight of w_n is used on the accepted request n , and we only have $W - w_n$ weight left for the set S of remaining requests that we accept.

Explaining the Case If Item n is Selected

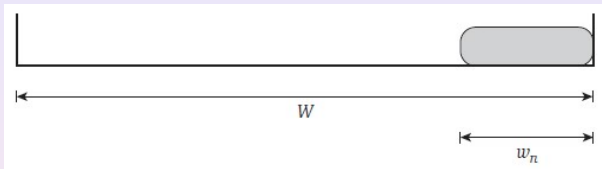


Figure: After item n is included in the solution, a weight of w_n is used up and there is $W - w_n$ available weight left.

Explaining the Case If Item n is Selected

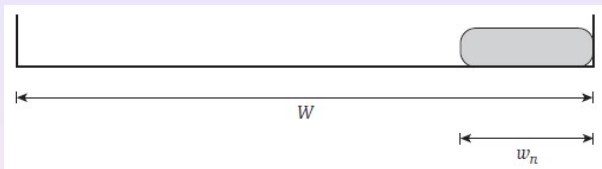


Figure: After item n is included in the solution, a weight of w_n is used up and there is $W - w_n$ available weight left.

Defining the Set of Sub-problems

- This suggests that we need more sub-problems: To find out the value for $OPT(n)$ we not only need the value of $OPT(n - 1)$, but we also need to know the best solution we can get using a subset of the first $n - 1$ items and total allowed weight $W - w_n$.

Explaining the Case If Item n is Selected

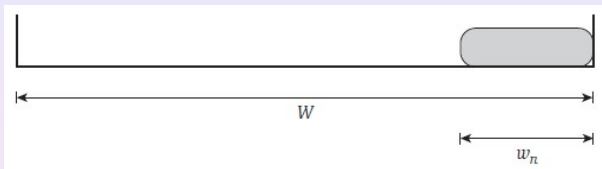


Figure: After item n is included in the solution, a weight of w_n is used up and there is $W - w_n$ available weight left.

Defining the Set of Sub-problems

- This suggests that we need more sub-problems: To find out the value for $OPT(n)$ we not only need the value of $OPT(n-1)$, but we also need to know the best solution we can get using a subset of the first $n-1$ items and total allowed weight $W - w_n$.
- We are therefore going to use many more sub-problems: one for each initial set $\{1, \dots, i\}$ of the items, and each possible value for the remaining available weight w .

Explaining the Case If Item n is Selected

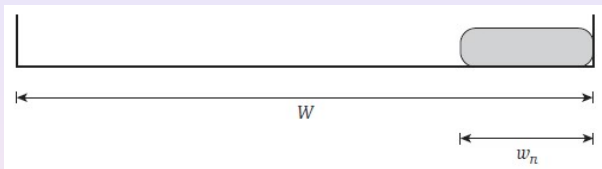


Figure: After item n is included in the solution, a weight of w_n is used up and there is $W - w_n$ available weight left.

Defining the Set of Sub-problems

- This suggests that we need more sub-problems: To find out the value for $OPT(n)$ we not only need the value of $OPT(n - 1)$, but we also need to know the best solution we can get using a subset of the first $n - 1$ items and total allowed weight $W - w_n$.
- We are therefore going to use many more sub-problems: one for each initial set $\{1, \dots, i\}$ of the items, and each possible value for the remaining available weight w .
- Assume that W is an integer, and all requests $i = 1, \dots, n$ have integer weights w_i . We will have a sub-problem for each $i = 0, 1, \dots, n$ and each integer $0 \leq w \leq W$.

Defining Sub-problems and Deriving Recurrence Linking Them

Explanation

- Let $OPT(i, w)$ denote the value of the optimal solution for the subset of the items $\{1, \dots, i\}$ with maximum allowed weight w , that is,

$$OPT(i, w) = \max_S \sum_{j \in S} w_j,$$

where the maximum is over subsets $S \subseteq \{1, 2, \dots, i\}$ that satisfy $\sum_{j \in S} w_j \leq w$.

Defining Sub-problems and Deriving Recurrence Linking Them

Explanation

- Let $OPT(i, w)$ denote the value of the optimal solution for the subset of the items $\{1, \dots, i\}$ with maximum allowed weight w , that is,

$$OPT(i, w) = \max_S \sum_{j \in S} w_j,$$

where the maximum is over subsets $S \subseteq \{1, 2, \dots, i\}$ that satisfy $\sum_{j \in S} w_j \leq w$.

- Using this new set of sub-problems, we will be able to express the value $OPT(i, w)$ as a simple expression in terms of values from smaller problems. Moreover, $OPT(n, W)$ is the quantity we're looking for in the end.

Defining Sub-problems and Deriving Recurrence Linking Them

Explanation

- Let $OPT(i, w)$ denote the value of the optimal solution for the subset of the items $\{1, \dots, i\}$ with maximum allowed weight w , that is,

$$OPT(i, w) = \max_S \sum_{j \in S} w_j,$$

where the maximum is over subsets $S \subseteq \{1, 2, \dots, i\}$ that satisfy $\sum_{j \in S} w_j \leq w$.

- Using this new set of sub-problems, we will be able to express the value $OPT(i, w)$ as a simple expression in terms of values from smaller problems. Moreover, $OPT(n, W)$ is the quantity we're looking for in the end.
- Let $\mathcal{O}$ denote an optimum solution for the original problem.

Defining Sub-problems and Deriving Recurrence Linking Them

Explanation

- Let $OPT(i, w)$ denote the value of the optimal solution for the subset of the items $\{1, \dots, i\}$ with maximum allowed weight w , that is,

$$OPT(i, w) = \max_S \sum_{j \in S} w_j,$$

where the maximum is over subsets $S \subseteq \{1, 2, \dots, i\}$ that satisfy $\sum_{j \in S} w_j \leq w$.

- Using this new set of sub-problems, we will be able to express the value $OPT(i, w)$ as a simple expression in terms of values from smaller problems. Moreover, $OPT(n, W)$ is the quantity we're looking for in the end.
- Let $\mathcal{O}$ denote an optimum solution for the original problem.
 - 1 If $n \notin \mathcal{O}$, then $OPT(n, W) = OPT(n-1, W)$, since we can simply ignore item n .

Defining Sub-problems and Deriving Recurrence Linking Them

Explanation

- Let $OPT(i, w)$ denote the value of the optimal solution for the subset of the items $\{1, \dots, i\}$ with maximum allowed weight w , that is,

$$OPT(i, w) = \max_S \sum_{j \in S} w_j,$$

where the maximum is over subsets $S \subseteq \{1, 2, \dots, i\}$ that satisfy $\sum_{j \in S} w_j \leq w$.

- Using this new set of sub-problems, we will be able to express the value $OPT(i, w)$ as a simple expression in terms of values from smaller problems. Moreover, $OPT(n, W)$ is the quantity we're looking for in the end.
- Let $\mathcal{O}$ denote an optimum solution for the original problem.
 - 1 If $n \notin \mathcal{O}$, then $OPT(n, W) = OPT(n-1, W)$, since we can simply ignore item n .
 - 2 If $n \in \mathcal{O}$, then $OPT(n, W) = w_n + OPT(n-1, W - w_n)$, since we now seek to use the remaining capacity of $W - w_n$ in an optimal way across items $1, 2, \dots, n-1$.
- Notice that when the n^{th} item is too big, that is, $W < w_n$, then we must have $OPT(n, W) = OPT(n-1, W)$.

Recurrence and Pseudocode

Using the same line of argument for the sub-problem for items $\{1, \dots, i\}$, and maximum allowed weight w , gives us the following recurrence.

Recurrence and Pseudocode

Using the same line of argument for the sub-problem for items $\{1, \dots, i\}$, and maximum allowed weight w , gives us the following recurrence.

Recurrence (6.8)

If $w < w_i$ then $OPT(i, w) = OPT(i - 1, w)$.

Otherwise $OPT(i, w) = \max(OPT(i - 1, w), w_i + OPT(i - 1, w - w_i))$.

Recurrence and Pseudocode

Using the same line of argument for the sub-problem for items $\{1, \dots, i\}$, and maximum allowed weight w , gives us the following recurrence.

Recurrence (6.8)

If $w < w_i$ then $OPT(i, w) = OPT(i - 1, w)$.

Otherwise $OPT(i, w) = \max(OPT(i - 1, w), w_i + OPT(i - 1, w - w_i))$.

```
Subset-Sum( $n, W$ )
```

```
  Array  $M[0 \dots n, 0 \dots W]$ 
```

```
  Initialize  $M[0, w] = 0$  for each  $w = 0, 1, \dots, W$ 
```

```
  For  $i = 1, 2, \dots, n$ 
```

```
    For  $w = 0, \dots, W$ 
```

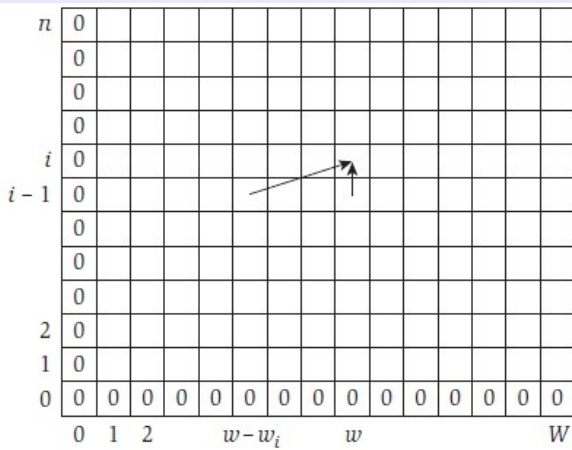
```
      Use the recurrence (6.8) to compute  $M[i, w]$ 
```

```
    Endfor
```

```
  Endfor
```

```
  Return  $M[n, W]$ 
```

Computation of the Entries of the Table



n	0														
	0														
	0														
	0														
i	0														
$i - 1$	0														
	0														
	0														
	0														
2	0														
1	0														
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	1	2			$w - w_i$		w							W

Figure: The two-dimensional table of OPT values. The leftmost column and bottom row is always 0. The entry for $OPT(i, w)$ is computed from the two other entries $OPT(i - 1, w)$ and $OPT(i - 1, w - w_i)$, as indicated by the arrows.

Running Time Analysis and Finding an Optimal Set of Items

Explanation

- The Subset-Sum(n, W) Algorithm correctly computes the optimal value of the problem, and runs in $O(n \cdot W)$ time.

Running Time Analysis and Finding an Optimal Set of Items

Explanation

- The Subset-Sum(n, W) Algorithm correctly computes the optimal value of the problem, and runs in $O(n \cdot W)$ time.
- Note that this method is not as efficient as our dynamic program for the Weighted Interval Scheduling problem, or the Rod Cutting problem.

Running Time Analysis and Finding an Optimal Set of Items

Explanation

- The Subset-Sum(n, W) Algorithm correctly computes the optimal value of the problem, and runs in $O(n \cdot W)$ time.
- Note that this method is not as efficient as our dynamic program for the Weighted Interval Scheduling problem, or the Rod Cutting problem.
- Indeed, its running time is not a polynomial function of n ; rather, it is a polynomial function of n and W , the largest integer involved in defining the problem. We call such algorithms pseudo-polynomial.

Running Time Analysis and Finding an Optimal Set of Items

Explanation

- The Subset-Sum(n, W) Algorithm correctly computes the optimal value of the problem, and runs in $O(n \cdot W)$ time.
- Note that this method is not as efficient as our dynamic program for the Weighted Interval Scheduling problem, or the Rod Cutting problem.
- Indeed, its running time is not a polynomial function of n ; rather, it is a polynomial function of n and W , the largest integer involved in defining the problem. We call such algorithms pseudo-polynomial.
- Pseudo-polynomial algorithms can be reasonably efficient when the numbers involved in the input are reasonably small; however, they become less practical as these numbers grow large.

Running Time Analysis and Finding an Optimal Set of Items

Explanation

- The Subset-Sum(n, W) Algorithm correctly computes the optimal value of the problem, and runs in $O(n \cdot W)$ time.
- Note that this method is not as efficient as our dynamic program for the Weighted Interval Scheduling problem, or the Rod Cutting problem.
- Indeed, its running time is not a polynomial function of n ; rather, it is a polynomial function of n and W , the largest integer involved in defining the problem. We call such algorithms pseudo-polynomial.
- Pseudo-polynomial algorithms can be reasonably efficient when the numbers involved in the input are reasonably small; however, they become less practical as these numbers grow large.
- To recover an optimal set S of items, we can trace back through the array M by a procedure similar to those we developed earlier.

Running Time Analysis and Finding an Optimal Set of Items

Explanation

- The Subset-Sum(n, W) Algorithm correctly computes the optimal value of the problem, and runs in $O(n \cdot W)$ time.
- Note that this method is not as efficient as our dynamic program for the Weighted Interval Scheduling problem, or the Rod Cutting problem.
- Indeed, its running time is not a polynomial function of n ; rather, it is a polynomial function of n and W , the largest integer involved in defining the problem. We call such algorithms pseudo-polynomial.
- Pseudo-polynomial algorithms can be reasonably efficient when the numbers involved in the input are reasonably small; however, they become less practical as these numbers grow large.
- To recover an optimal set S of items, we can trace back through the array M by a procedure similar to those we developed earlier.
- Given a table M of the optimal values of the sub-problems, the optimal set S can be found in $O(n)$ time.

Extension: The Knapsack Problem

Dynamic Programming Solution for the Knapsack Problem

- Recall that in the Knapsack problem, each item i has a nonnegative weight w_i as before, and also a distinct value v_i . Our goal is to find a subset S of maximum value $\sum_{i \in S} v_i$, subject to the restriction that the total weight of the set should not exceed W : $\sum_{i \in S} w_i \leq W$.

Extension: The Knapsack Problem

Dynamic Programming Solution for the Knapsack Problem

- Recall that in the Knapsack problem, each item i has a nonnegative weight w_i as before, and also a distinct value v_i . Our goal is to find a subset S of maximum value $\sum_{i \in S} v_i$, subject to the restriction that the total weight of the set should not exceed W : $\sum_{i \in S} w_i \leq W$.
- It is not hard to extend our dynamic programming algorithm to this more general problem.

Extension: The Knapsack Problem

Dynamic Programming Solution for the Knapsack Problem

- Recall that in the Knapsack problem, each item i has a nonnegative weight w_i as before, and also a distinct value v_i . Our goal is to find a subset S of maximum value $\sum_{i \in S} v_i$, subject to the restriction that the total weight of the set should not exceed W : $\sum_{i \in S} w_i \leq W$.
- It is not hard to extend our dynamic programming algorithm to this more general problem.
- We use the analogous set of sub-problems, $OPT(i, w)$, to denote the value of the optimal solution for the subset of the items $\{1, \dots, i\}$ and maximum available weight w .

Extension: The Knapsack Problem

Dynamic Programming Solution for the Knapsack Problem

- Recall that in the Knapsack problem, each item i has a nonnegative weight w_i as before, and also a distinct value v_i . Our goal is to find a subset S of maximum value $\sum_{i \in S} v_i$, subject to the restriction that the total weight of the set should not exceed W : $\sum_{i \in S} w_i \leq W$.
- It is not hard to extend our dynamic programming algorithm to this more general problem.
- We use the analogous set of sub-problems, $OPT(i, w)$, to denote the value of the optimal solution for the subset of the items $\{1, \dots, i\}$ and maximum available weight w .
- We consider an optimal solution $\mathcal{O}$, and identify two cases depending on whether or not $n \in \mathcal{O}$.

Extension: The Knapsack Problem

Dynamic Programming Solution for the Knapsack Problem

- Recall that in the Knapsack problem, each item i has a nonnegative weight w_i as before, and also a distinct value v_i . Our goal is to find a subset S of maximum value $\sum_{i \in S} v_i$, subject to the restriction that the total weight of the set should not exceed W : $\sum_{i \in S} w_i \leq W$.
- It is not hard to extend our dynamic programming algorithm to this more general problem.
- We use the analogous set of sub-problems, $OPT(i, w)$, to denote the value of the optimal solution for the subset of the items $\{1, \dots, i\}$ and maximum available weight w .
- We consider an optimal solution $\mathcal{O}$, and identify two cases depending on whether or not $n \in \mathcal{O}$.
 - If $n \notin \mathcal{O}$, then $OPT(n, W) = OPT(n-1, W)$.

Extension: The Knapsack Problem

Dynamic Programming Solution for the Knapsack Problem

- Recall that in the Knapsack problem, each item i has a nonnegative weight w_i as before, and also a distinct value v_i . Our goal is to find a subset S of maximum value $\sum_{i \in S} v_i$, subject to the restriction that the total weight of the set should not exceed W : $\sum_{i \in S} w_i \leq W$.
- It is not hard to extend our dynamic programming algorithm to this more general problem.
- We use the analogous set of sub-problems, $OPT(i, w)$, to denote the value of the optimal solution for the subset of the items $\{1, \dots, i\}$ and maximum available weight w .
- We consider an optimal solution $\mathcal{O}$, and identify two cases depending on whether or not $n \in \mathcal{O}$.
 - If $n \notin \mathcal{O}$, then $OPT(n, W) = OPT(n-1, W)$.
 - If $n \in \mathcal{O}$, then $OPT(n, W) = v_n + OPT(n-1, W - w_n)$.
- Using this recurrence, we can write down a completely analogous dynamic programming algorithm, that runs in $O(n \cdot W)$ time.